

Vibe Coding从入门到精通

01 为什么用Vibe Coding

AI编程方式在最近三年里变了三次。搞清楚这个演变路径，你就能理解为什么现在开发需要使用VibeCoding。

三年变了三次

- 2022年，GitHub Copilot出来了。你写上半句，它帮你猜下半句。像一个坐在旁边的实习生，打字确实快了，但本质没变：你还是那个写代码的人。
- 2023到2024年，Cursor火了。你可以用自然语言让编辑器帮你改函数、重构模块，不用精确描述「怎么写」，说「我想要什么效果」就行。后来Cursor也加了Agent模式，能跨文件操作、自动跑命令。但它始终长在IDE里，是编辑器的延伸。
- 2025年，各类开发CLI出来了，比如Claude Code、OpenCode、WorkBuddy。它不住在任何编辑器里，直接在终端运行。你描述一个需求，它自己规划步骤、读代码、写代码、跑测试、操作git，整个循环自动完成。你的角色从「写代码的人」变成了「给指令的人」。



三步走下来，变的不是技术有多先进，而是你和AI之间的关系。Copilot是你的输入法，Cursor是你的结对伙伴，Agent是你的独立工程师团队。

它跟平台AI辅助到底有什么不一样

这是被问得最多的问题：「我使用deepseek网页，不都是AI帮我写代码吗？」

确实，2025年之后的各个AI平台已经很强了，能跨文件操作、理解整个项目、自动执行命令。

如何指挥这些AI工具开发一个可以实际发布的产品，而不是一个玩具或者demo的区别就在于，你是如何和AI交互，交互的编码的过程就是vibecoding，vibe coding里能不能用好，关键就是掌握vibe coding基础理论，总结过程中的经验，学习使用的方法论。

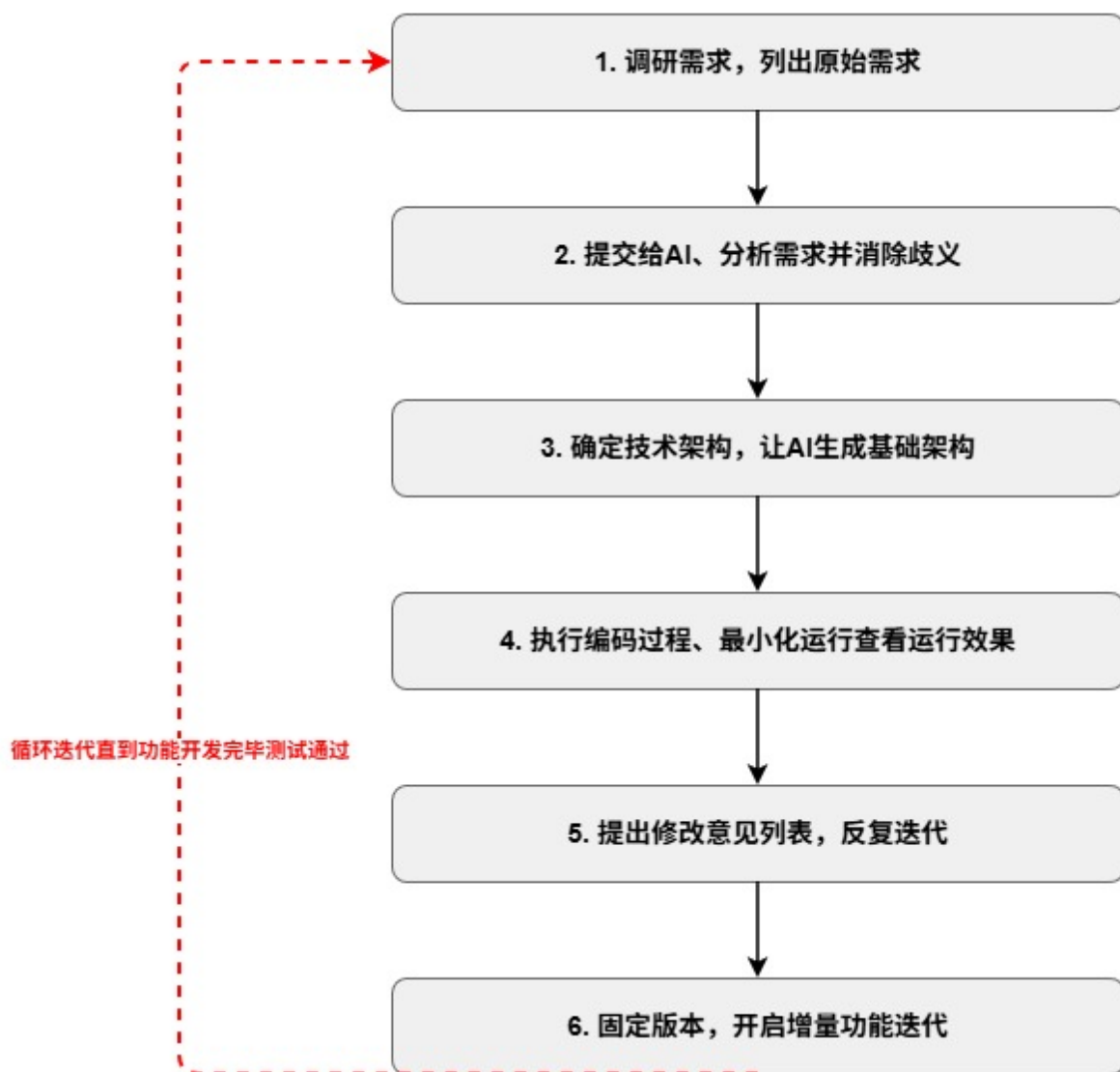
*人和人之间沟通时彼此都存在不能理解对方意图的情况，何况对AI说清楚你的意图，所以学习如何使用vibecoding，不是学习如何对AI说话，而是学习如何把你的意图表述清晰，无歧义，边界清楚。

我的经验：现在开发我本人从未手写过代码，所有产品都是用AI完成的。AI辅助编码让「不会写代码但能构建产品」这件事，从少数人的实验变成了大多数人的可能。

我们使用vibecoding其实不是在帮你写代码，而是构建产品

一个场景分析：

终端Agent（推荐用codebuddy）：「帮我做一个支持Markdown的博客系统，用Next.js，部署到Vercel，支持暗色模式和RSS。」它会：分析需求→选技术方案→创建项目→逐步实现→跑测试→修bug→完成。全程你需要通过vibe coding方式确认和调整方向，给出修改意见。



该课程给你的提升

工程师，想提高10倍效率。你已经会写代码，但每天大量时间花在样板代码、调试、写测试、处理CI/CD上。使用AI vibecoding开发能接管这些，让你把精力放在架构决策和产品思考上。

产品经理，想自己做MVP。你有产品直觉和用户洞察，但受限于开发资源。AI开发让你一个周末做出一个能跑的原型，不用等排期，不用写PRD等开发理解。

创业者，想实现一人公司。你想验证商业想法，但不想在技术上花太多钱和时间。AI开发让一个人拥有一个小团队的开发能力，网站、App、后端API，都可以一个人搞定。

核心建议

不管你属于哪一类，这课程假设你聪明也很主动，但可能从没用过AI编程工具。我们从零开始，但不会在基础概念上磨叽太久。

02 10分钟完成AI平台安装

可以选择的AI开发平台很多，在本次实训中，我们推荐使用

腾讯旗下codebuddy，下载地址 <https://www.codebuddy.ai/ide>

字节旗下Trae，下载地址 <https://www.trae.cn>

*注意 一个开发小组里使用的AI开发平台需要一致。

为什么选择这2个平台，其实很多同学肯定也使用过其他开发平台，比如Cursor、Claude Code。

第一、我们现在开发产品无法转换收入而是在学习阶段，所以选择学习选择免费平台或者价格具备性价比的平台。

第二、使用国内的平台和基座模型

按照完成codebuddy（以下课程都以该平台为例子）

我们还需要安装必备的软件，AI开发平台常用开发工具列表

📋 工具清单总表

序号	工具名称	推荐的稳定版本	主要作用
1	Git	2.40+	版本控制系统，代码管理、分支管理、协作开发
2	Node.js	22 LTS以上版本	JavaScript运行时环境，用于前端开发和服务器端应用
3	Python	3.11	高级编程语言，AI/ML开发、数据分析、自动化脚本
4	OpenJDK	21 LTS	Java开发工具包，Java应用开发和编译

安装这些工具，不需要再额外出说明，AI工具可以帮我安装，在codebuddy里面输入提示词：

🔗 各工具安装提示词

1. Git 安装提示词

请帮我安装Git版本控制系统：

- 操作系统：Linux（Ubuntu/Debian）

- 安装版本：2.40或更高稳定版

- 配置Git用户信息（用户名和邮箱）

- 安装完成后验证git --version

2. Node.js 安装提示词

请帮我安装Node.js运行时环境：

- 操作系统：Linux

- 推荐版本：Node.js 22 LTS（长期支持版本）

- 同时安装npm包管理器（最新稳定版）

- 配置淘宝镜像源加速国内下载

- 验证安装：node --version 和 npm --version

3. Python 安装提示词

请帮我安装Python编程语言：

- 操作系统：Linux

- 推荐版本：Python 3.11

- 包含pip包管理工具

- 配置pip国内镜像源（豆瓣或清华）

- 创建Python虚拟环境工具venv

- 验证：python3 --version

4. OpenJDK 安装提示词

请帮我安装OpenJDK Java开发工具包：

- 操作系统：Linux
- 推荐版本：OpenJDK 21 LTS 或 17 LTS
- 安装JDK（包含JRE开发环境）
- 配置JAVA_HOME环境变量
- 验证：java --version 和 javac --version

关于token额度的事情

现阶段2026-5, 这2个AI开发平台的额度每天限量免费，首次启动时会自动弹浏览器让你登录或注册即可

03 你的第一个项目

做个什么

一个每日新闻聚合器，CLI工具。功能很简单：

- 从几个RSS源,抓取最新文章
- 用AI总结每篇文章的要点
- 输出一份格式整齐的Markdown 日报

为什么选这个？够小，一个下午能做完。又够完整，涉及网络请求、数据处理、AI调用、文件输出。一次项目就能把AI开发平台的各种能力体验个遍。

先转换一下心态：从现在开始，你是产品经理，AI Agent是你的工程师。你的活是说清楚要什么，不是写代码。

哪怕你是资深程序员，也请先按这个方式来一遍

第一步：告诉AI你要什么

新建一个空文件夹，打开codebuddy，打开空文件夹，点击右边对话框。

然后，用自然语言告诉AI你想要什么：

你输入的prompt：

帮我做一个新闻聚合CLI工具。需求如下：

1 . 从以下RSS源抓取最近24小时的文章：

- TechCrunch AI (<https://techcrunch.com/category/artificial-intelligence/feed/>)
- The Verge AI (<https://www.theverge.com/rss/ai-artificial-intelligence/index.xml>)
- Hacker News前30条(<https://hnrss.org/newest?q=AI&count=30>)

2 . 对每篇文章提取标题、链接、发布时间、来源

3 . 按时间倒序排列，输出一份Markdown格式的日报到output/ 目录

4 . 用TypeScript写，用tsx直接运行

先别急着写代码，给我一个实现方案。

注意最后那句「先别急着写代码，给我一个实现方案」。这不是客气，是一个重要技巧：让AI先想清楚再动手。

*当然你也可以通过切换cra和plan来控制

核心建议

需求描述的质量直接决定输出质量。几个原则：说清楚最终产物长什么样（Markdown 日报）、指定技术选型（TypeScript + tsx）、给具体数据源URL而不是说「几个RSS源」。越具体越好。

第二步：看它怎么想的

AI收到需求后，不会直接开写，会先给你一个方案。大概长这样：

我来帮你做这个AI新闻聚合工具。先说一下方案：

项目结构：

```
ai-news-digest/
├── src/
│   ├── index .ts          # 入口文件
│   ├── fetcher .ts        # RSS抓取逻辑
│   ├── formatter .ts      # Markdown格式化
│   └── types .ts           # 类型定义
├── output/                 # 日报输出目录
├── package .json
└── tsconfig .json
```

技术方案：

- RSS解析用rss-parser 库
- HTTP请求用node内置的fetch
- 日期处理用date-fns
- 用tsx直接运行TypeScript，不需要编译步骤

实现流程：

- 1 . 并行抓取三个RSS源
- 2 . 统一格式、过滤24小时内文章
- 3 . 按时间排序
- 4 . 生成Markdown格式日报
- 5 . 写入output/YYYY-MM-DD.md

需要我开始实现吗？

这时候你就是在看工程方案的产品经理。觉得行就说OK，想调整就直接说。比如：

你的反馈：

方案没问题。补充两点：

1. 日报里每篇文章除了标题和链接，加一句话摘要（从文章description里截取前100字）
2. 日报开头加一个统计：共收录X篇，来自Y个源

这个来回就是vibe coding 编程的核心：AI出方案，你补充细节，AI修正。不用画流程图、不用写技术文档，直接用人话说。

第三步：看它干活

确认方案后，AI Agent开始执行。你会看到一系列操作：

1、初始化项目

运行npm init -y，然后安装依赖包。你会看到它请求运行命令的权限提示：

2、创建源代码文件

会逐个创建TypeScript文件。你会看到它写入代码的过程，每个文件都会展示差异（diff）。不需要逐行看，但可以快速扫一眼文件结构是否合理

3、运行测试

代码写完后，AI通常会自己试着运行一次看看有没有报错。如果报错了，它会自己读错误信息、找问题、修代码、再运行，形成一个自动修复循环。这里的报错指的是运行时的报错，而不是功能错误。

整个过程大约2-5分钟。你干嘛？看着就行。就像把任务交给新同事，前几次你会盯着看他怎么做事。等熟悉了他的风格，以后放心让他自己干就好。

第四步：看看结果对不对

code buddy干完了，跑一下看看：

```
npx tsx src/index .ts
```

如果一切顺利，你会在output/目录下看到一个Markdown文件。

打开文件看看格式和内容。大多数情况下，第一次就能跑通。如果报错了？直接把错误信息丢给AI：

运行报错时，把错误信息粘贴给Claude：

运行报错了：

```
TypeError: Cannot read properties of undefined (reading 'map')
at formatArticles (src/formatter.ts:15:23)
```

AI agent会读错误信息、定位问题、改代码、再跑。这个报错到修复的循环，一般1-2轮就搞定。

第五步：加功能

能跑了，但你想加点东西。继续用自然语言说就行：

第一个改进：加AI摘要

现在每篇文章的摘要是从description里截取的，比较粗糙。

改成用AI来总结：对每篇文章的标题+description，用deepseek API生成一句话总结。

API key从环境变量 deepseek_API_KEY 读取。

AI会修改代码，加入API调用逻辑。deepseek需要你自己去官网申请

你验证后继续：

第二个改进：加定时运行

加一个cron模式，每天早上8点自动运行一次，用node-cron实现。

加一个命令行参数：

- `npx tsx src/index .ts` 立即运行一次
- `npx tsx src/index .ts -cron` 开启定时模式

第三个改进：加去重逻辑

有些文章在多个源里重复出现了。加一个基于URL的去重。

每一轮，改代码、跑测试、确认结果。你始终只做两件事：说清楚要什么，验证结果。

到这里，第一个项目完成了。回头看看整个过程：



不管项目大小，不管是小工具还是完整产品，底层模式都是这五步。

一个重要的心态转变

做完这个项目，你应该有个直觉上的感受：你的价值不在于写代码，而在于定义要做什么、判断做得对不对。

很多工程师第一次用vibe coding 时，本能地想看每一行代码、理解每个实现细节。正常反应，但它会让你变慢。

更高效的方式是把它当团队成员来管理：(后续会带领大家打造自己的AI Agent开发团队)

传统编程	Vibe Coding编程
自己想方案，自己写代码	描述需求，AI出方案和代码
一行一行调试	把错误信息给AI，它自己调
查文档、查接口	直接问AI「怎么实现XXX」
代码review靠人工	让AI解释它写的代码
重构要先理解全部代码	告诉AI「把这块重构成XXX模式」

不是说完全不管代码。而是你的注意力应该在更高层面：需求准不准确？方案合不合理？结果符不符合预期？这些才是你该花时间的地方。

04 新手常见困惑

1.[代码看不懂怎么办？]

直接问它。「解释一下fetcher.ts 的实现逻辑」，code buddy会用人话讲清楚。还可以追问：「为什么用 Promise.allSettled 而不是Promise.all？」它会解释背后的技术选择。

你不需要能写出这段代码，但需要理解它在做什么。就像你不用会修发动机，但得知道车在正常运转。

2.[写错了怎么办？]

直接说哪里不对。你不需要知道怎么改，描述现象就够了：

推荐:

直接说哪里不对。你不需要知道怎么改，描述现象就够了：

运行后只输出了TechCrunch的文章，另外两个源的文章没有。检查一下抓取逻辑。

不推荐

「你的代码第23行有bug。」（除非你确实知道问题在哪）

描述现象比定位代码行更有效。AI可能发现问题根源不在你以为的地方。

3.「该管多少？」

四个字：信任但验证。让AI去做，但每一步检查结果。方案阶段仔细看，确保方向对。编码阶段扫一眼文件结构就行。运行阶段看输出是否符合预期。

改进阶段多测试边界情况：空数据、网络超时、格式异常。

这个分寸感做几个项目自然就有了。

4.「跑偏了怎么办？」

偏得不远，直接纠正：「停，不要用XXX库，换YYY。」偏得太远，按Esc 停止，重新描述需求。

撤销修改：直接给AI说，撤销上一次的所有修改

一个经验：纠正两次还不行，就果断停下来重来。在错误基础上打补丁只会越补越乱。

总结：这一章的核心-描述需求、审查方案、确认执行、验证结果、迭代改进。你管要什么和好不好，AI管怎么实现。这个分工形成默契之后，生产力会有质的变化。